IoT 1

Les 7 – I²C, State machines,…

I²C

I²C

- **Wat**?
  - **I**nter **I**ntegrated **C**ircuit
  - Ontwikkeld door Philips.
  - Digitale multi-master multi-slave **synchrone communicatiebus.**
  - Meestal voor communicatie **op eenzelfde print**.
  - Sowieso **afstand** proberen te **beperken** tot minder dan één meter.

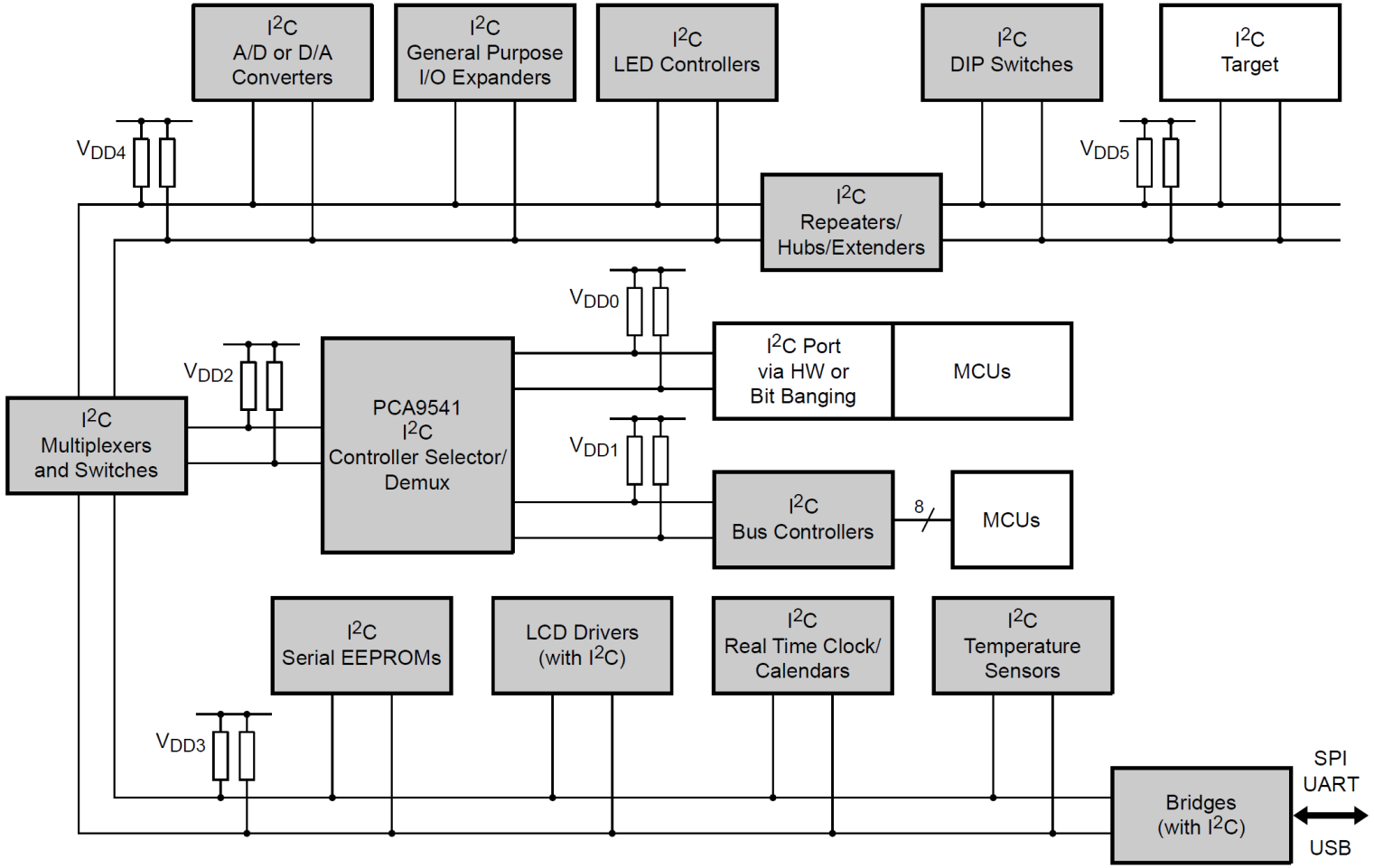
Bron: https://en.wikipedia.org/wiki/I%C2%B2C#/media/File:I%C2%B2C_bus_logo.svg

I²C

- **Hoe**?
  - Twee draden nodig: SDA (**S**erial **DA**ta) en SCL (**S**erial **CL**ock).
  - Moeten via **pull-up** naar de gewenste spanning ‘opgetrokken’ worden.
  - Houd rekening met de **voedingsspanning** van de devices: 3,3V, 5V, …
  - Indien nodig, gebruik ‘**level shifters**’.
  - Maximum **deelnemers** is beperkt door: aantal **adressen** en maximum **buscapaciteit**.
  - Werkt typisch op **100kbit/s**.

I²C

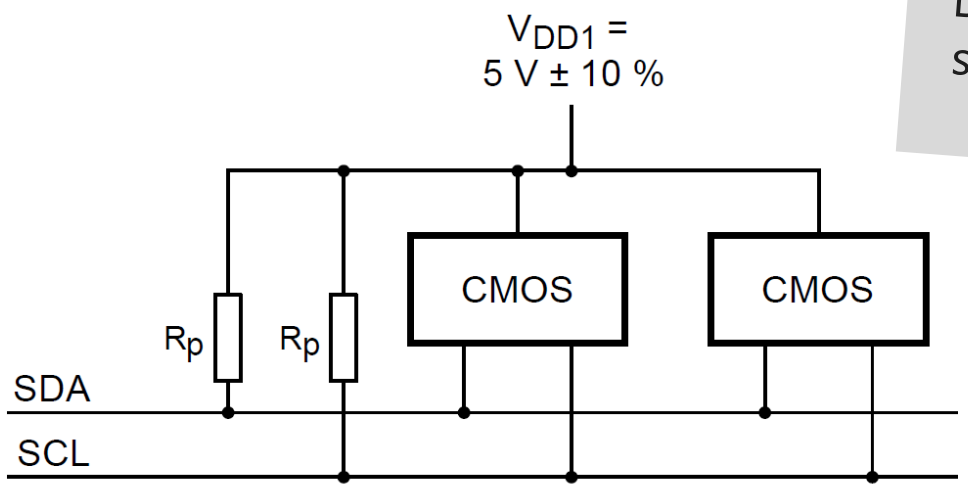
- Typische (complexe) **opstelling**



Bron: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

I²C

- Typische (eenvoudige) **opstelling**



OPM: een veelgebruikte waarde voor de pull-up weerstanden is **4,7kOhm**.

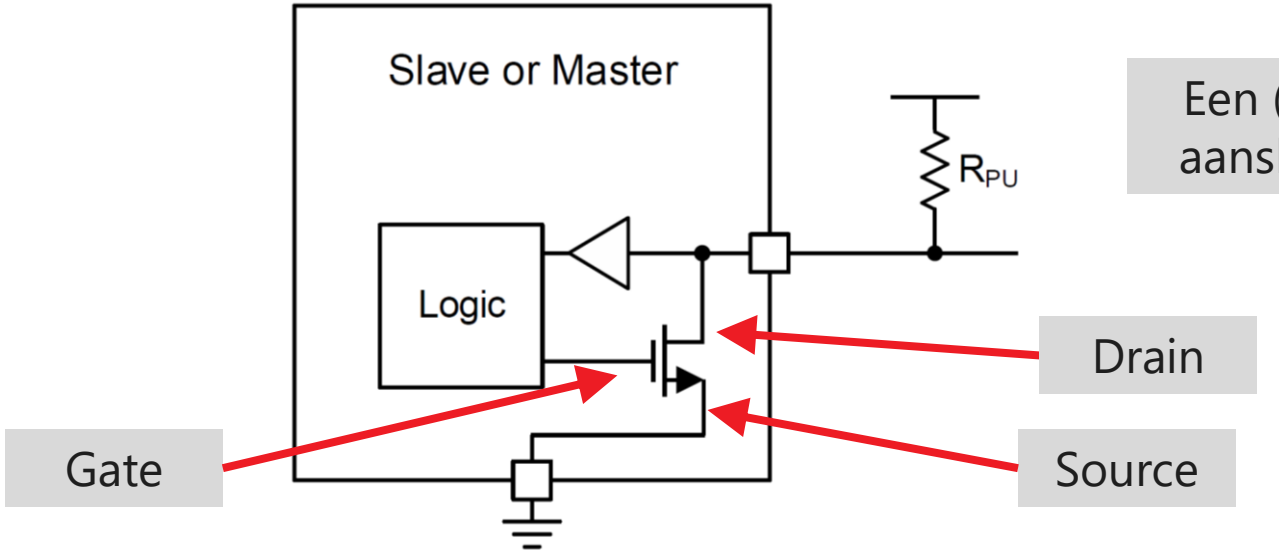
Dat geeft het goed evenwicht tussen stroombeperking en ongevoeligheid aan storingen...

Complementary **M**etal–**O**xide–**S**emiconductor is een type fabricageproces, dat complementaire paren P-type en N-type MOSFET's gebruikt, voor het bouwen van chips en logische functies in die chips.

Bron: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

I²C

- **Waarom** werken met **pull-ups**?
  - Omdat je op die manier verschillende devices '**parallel**' kan zetten **zonder** dat ze elkaar fysische **schade** kunnen toebrengen.
  - De chips werken dan immers met '**open drain**' uitgangen.

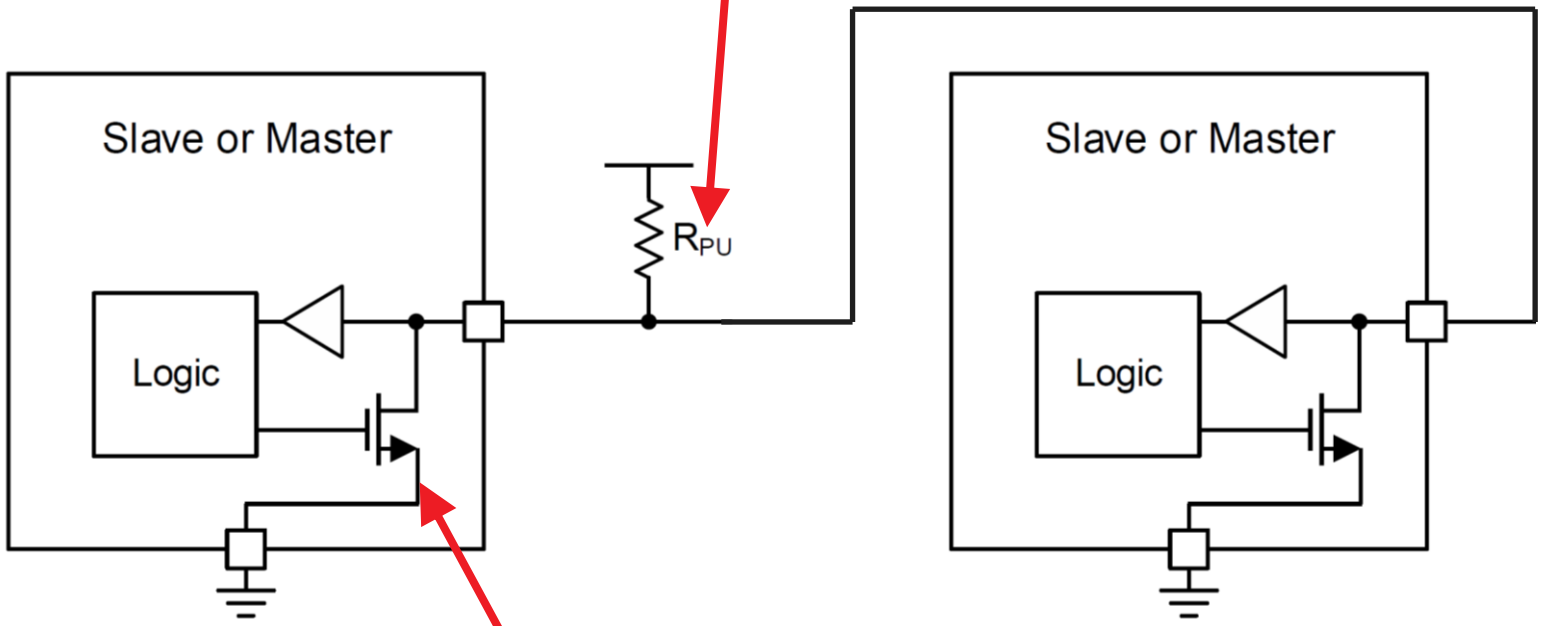


Een (**MOS)FET** (transistor) heeft drie aansluitingen: Drain, Gate en Source.

Bron: https://www.ti.com/lit/pdf/slva704

I²C

Pull-up weerstand die de bus ‘hoog’ trekt, als alle FET’s ‘open’ staan.



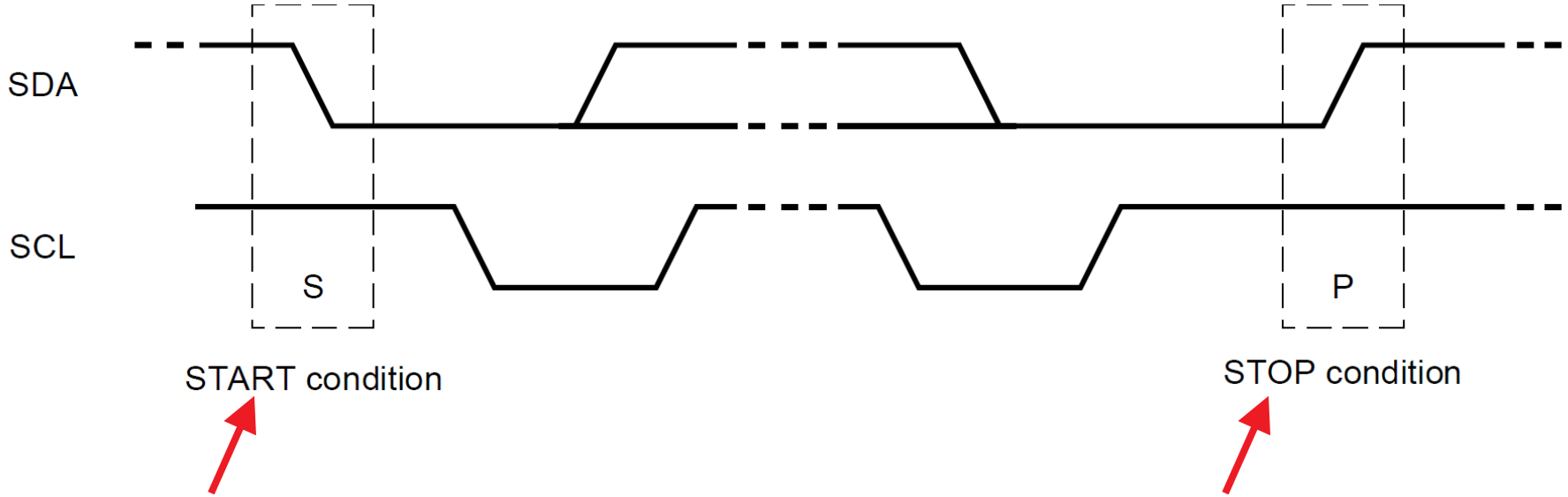
FET (transistor) die werkt als schakelaar (aan/uit).

Denk na: wat gebeurt er als alle FET’s open staan? Als er één dicht is? Als beiden dicht zijn?

Bron: https://www.ti.com/lit/pdf/slva704

I²C

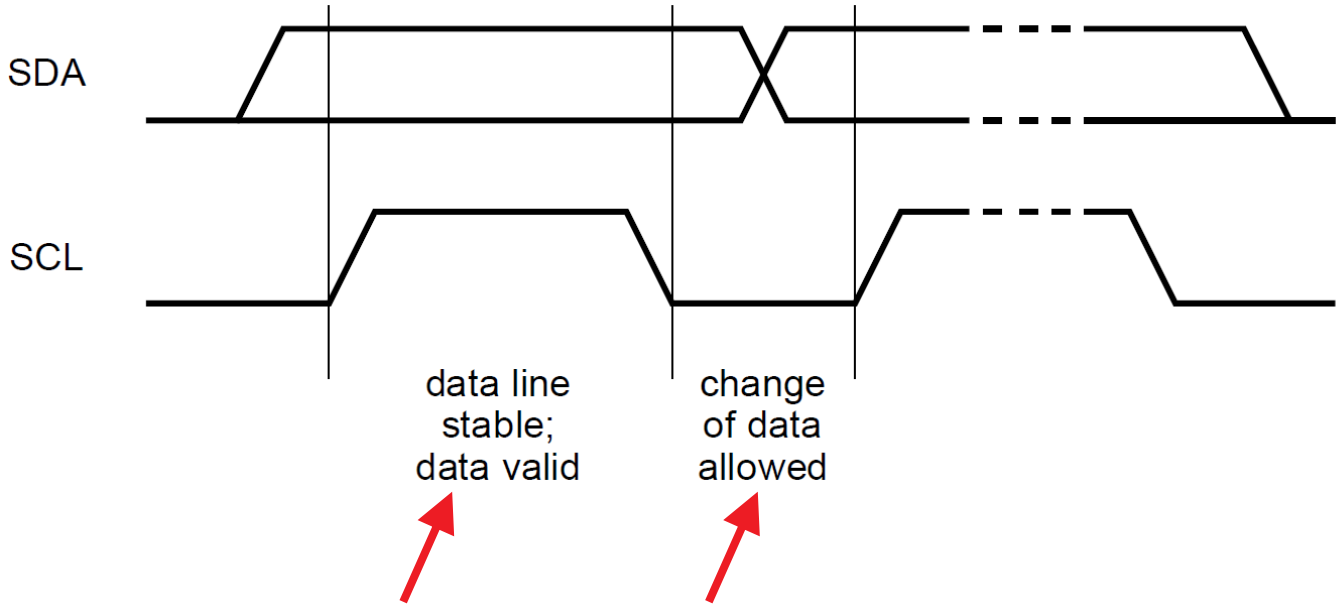
- **Afspraken**
  - Omdat de bus bidirectioneel is, moeten er **afspraken** gemaakt worden **i.v.m. het versturen** van data.



Bron: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

I²C

- **Afspraken**
  - Omdat de bus bidirectioneel is, moeten er **afspraken** gemaakt worden **i.v.m. het versturen** van data.



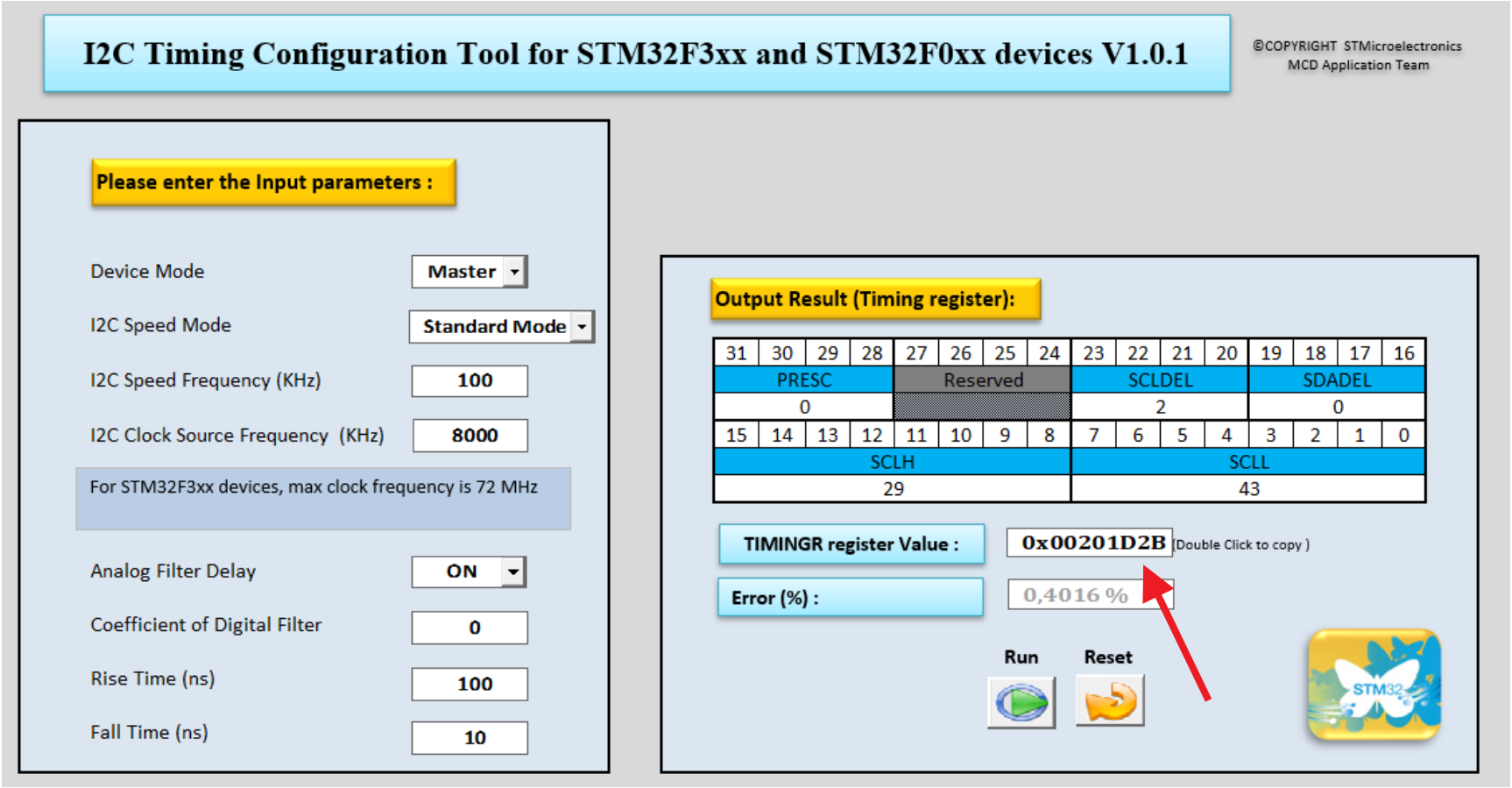
Bron: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

I²C

- En bij onze **STM32F091RC**?
    - Die heeft **twee** I²C-modules…
    - Timing gebeurt in het bijhorende register (**TIMINGR**).
    - De berekening voor de instelling van die registers, kan manueel of **via** een application note (AN4235) en **Excel**.
    - Eénmaal ingesteld, bijna altijd **dezelfde waarde behouden**…

I²C

- En bij onze **STM32F091RC**?



I²C

- **Instellingen** voor het gebruik van I²C

1. Pinnen instellen als **alternate function**.
2. I²C-module van **klok** voorzien.
3. **Timing** instellen (100kHz, rise and fall times, …) via Excel.
4. I²C-module **inschakelen**.
5. …

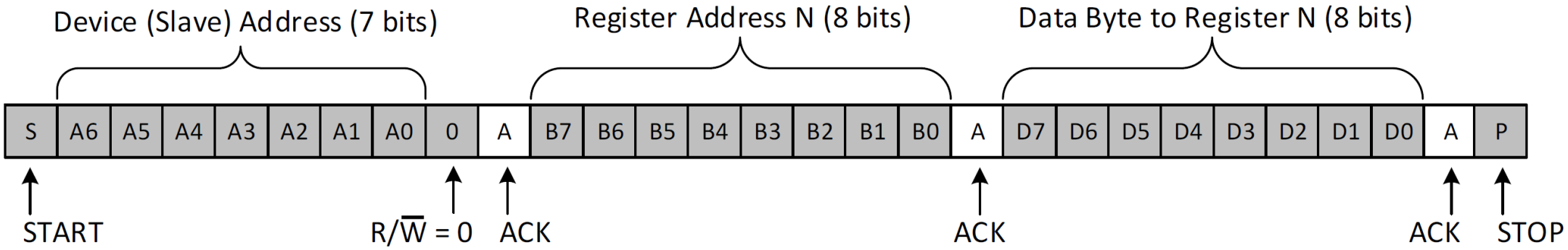
```
// Timing correct instellen (via AN4235 en Excel bestand van ST)
// 100kHz vertrekkende vanuit 8MHz (HSI)
I2C1->TIMINGR = 0x00201D2B;

// I²C1-module enable
I2C1->CR1 |= I2C_CR1_PE;
}
```

I²C

- Typisch **schrijven** naar een device

<u>Write to One Register in a Device</u>



R/$\overline{\mathbf{W}}$ = read/**write**

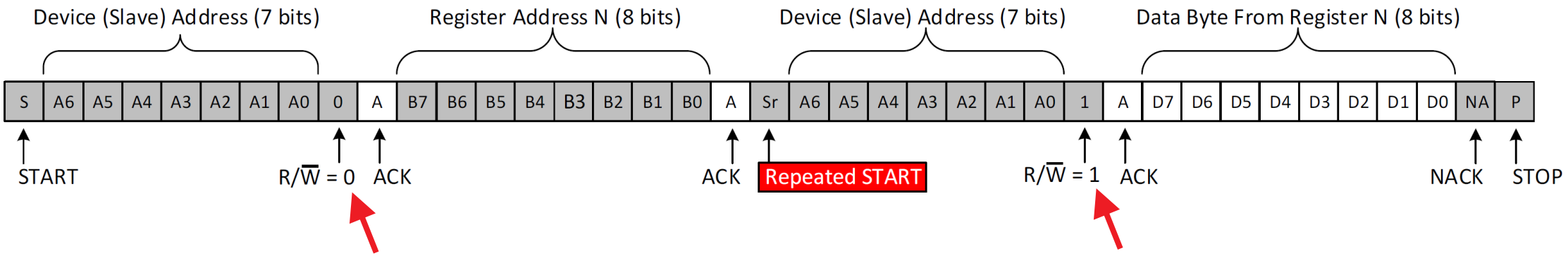
ACK = aknowledge

Bron: https://www.ti.com/lit/pdf/slva704

I²C

- Typisch **lezen** van een device

<u>Read From One Register in a Device</u>



NACK = not aknowledge

Bron: https://www.ti.com/lit/pdf/slva704

I²C

- **Firmware** voorbeeld

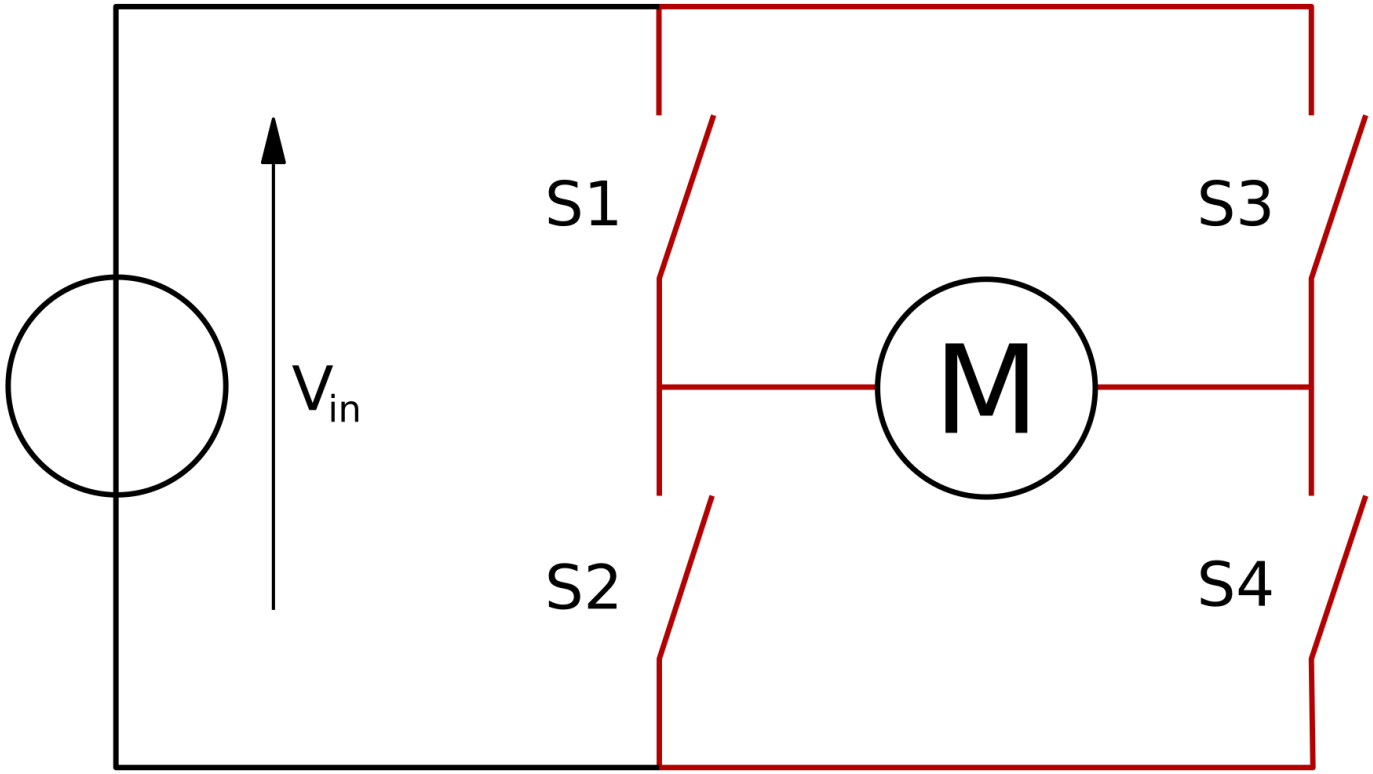
```
45    // Schrijfactie, ranging starten.
46    I2C1->CR2 &= ~I2C_CR2_RD_WRN;
47
48    // Slave address waarmee gecommuniceerd moet worden invullen.
49    I2C1->CR2 &= 0xFFFFFF00;                          // Slave address resetten.
50    I2C1->CR2 |= (SLAVE_ADDRESS << 1);                // Adres klaarzetten.
51
52    // Aantal te versturen bytes, waarna een stop gegenereerd moet worden (TC vlag wordt gezet).
53    I2C1->CR2 &= ~I2C_CR2_NBYTES;                     // NBYTES wissen.
54    I2C1->CR2 |= (2 << 16);                           // 2 bytes te versturen, dus 2 in NBYTES plaatsen.
55
56    // Start conditie.
57    I2C1->CR2 |= I2C_CR2_START;
58
59    // Bytes versturen.
60    while((I2C1->ISR & I2C_ISR_TXE) != I2C_ISR_TXE);  // Is verzendbuffer leeg?
61    I2C1->TXDR = 0;                                   // Register 0 versturen (command register).
62    while((I2C1->ISR & I2C_ISR_TXE) != I2C_ISR_TXE);  // Is verzendbuffer leeg?
63    I2C1->TXDR = 0x51;                                // Ranging in cm starten.
64
65    // Stop conditie.
66    while((I2C1->ISR & I2C_ISR_TC) != I2C_ISR_TC);    // Wacht op transmission complete.
67    I2C1->CR2 |= I2C_CR2_STOP;
```


hogeschool vives

I²C

- Toepassingsvoorbeeld: I²C **H-brug**

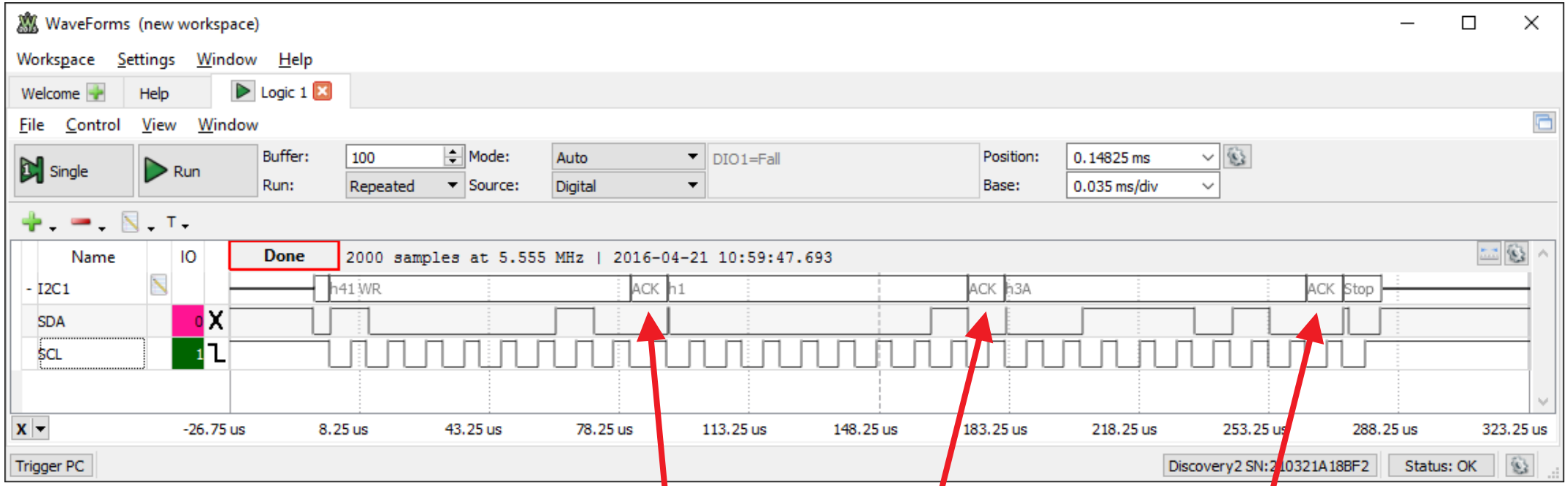
Met een H-brug kan je de **stroomzin omdraaien** doorheen een DC-motor... Ideaal om ook de draaizin te veranderen!



Bron: https://en.wikipedia.org/wiki/H-bridge#/media/File:H_bridge.svg

I²C

- Toepassingsvoorbeeld: I²C **H-brug**



Merk telkens de '**aknowledge**' op die de slave gebruikt **om teken van leven te geven aan de master**…

I²C

- Toepassingsvoorbeeld: I²C **H-brug**

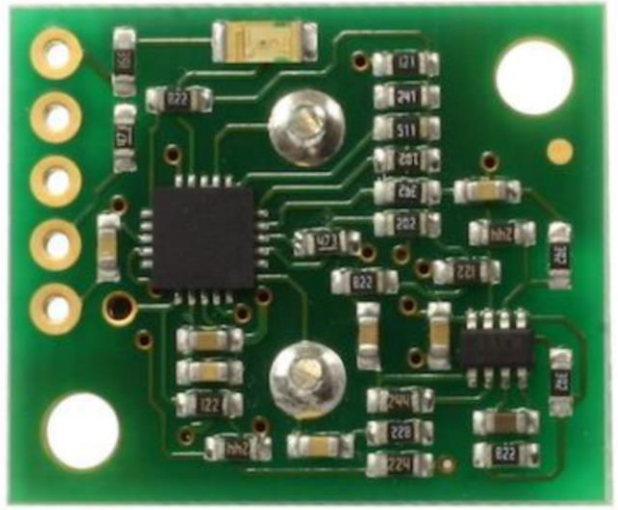
Bron: https://www.youtube.com/watch?v=6Zx_ZVKtiSE

hogeschool vives

I²C

- Toepassingsvoorbeeld: ultrasonic **range finder** (SRF-02)

Vergeet de **pull-up weerstanden** niet… Naar de 5V (bij voorkeur)!
PB8 (SCL1) en PB9 (SDA1) zijn 5V-tolerant. **Zoek dat op** in de datasheet!



Bron: https://www.robot-electronics.co.uk/htm/srf02techI2C.htm

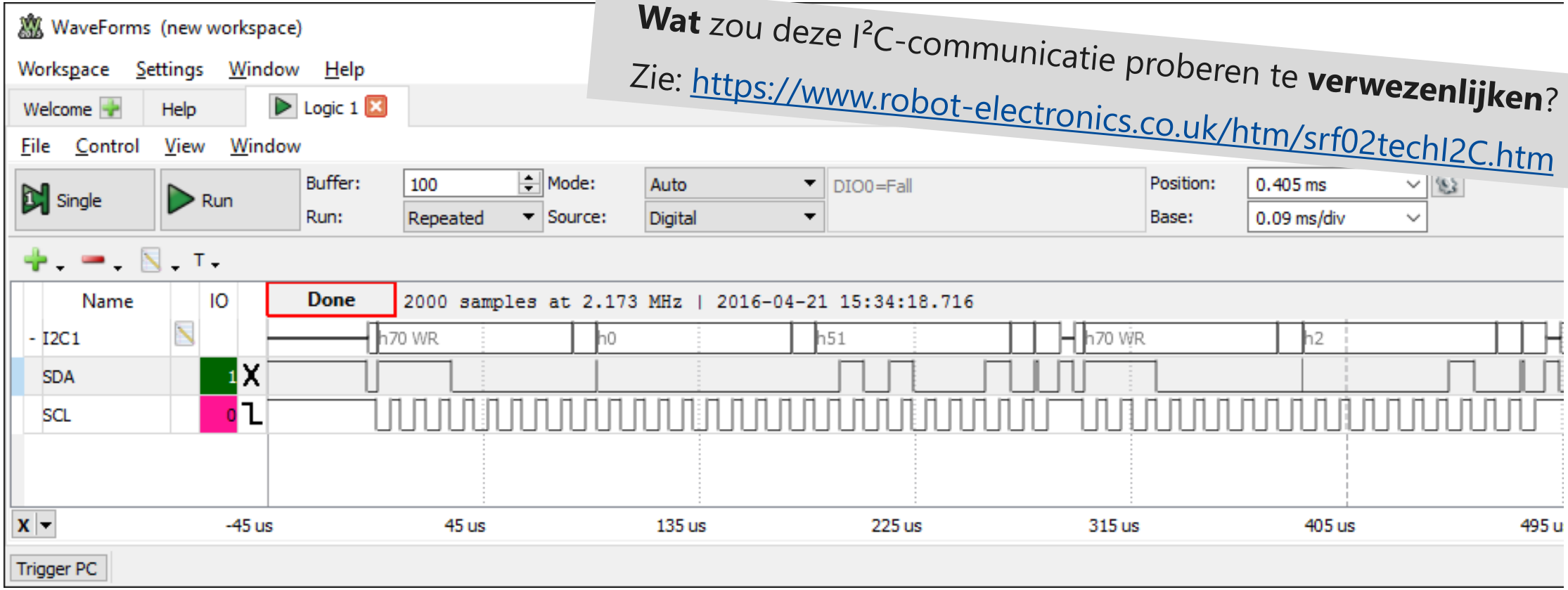
hogeschool vives

I²C

- Toepassingsvoorbeeld: ultrasonic **range finder** (SRF-02)

Wat zou deze I²C-communicatie proberen te **verwezenlijken**?

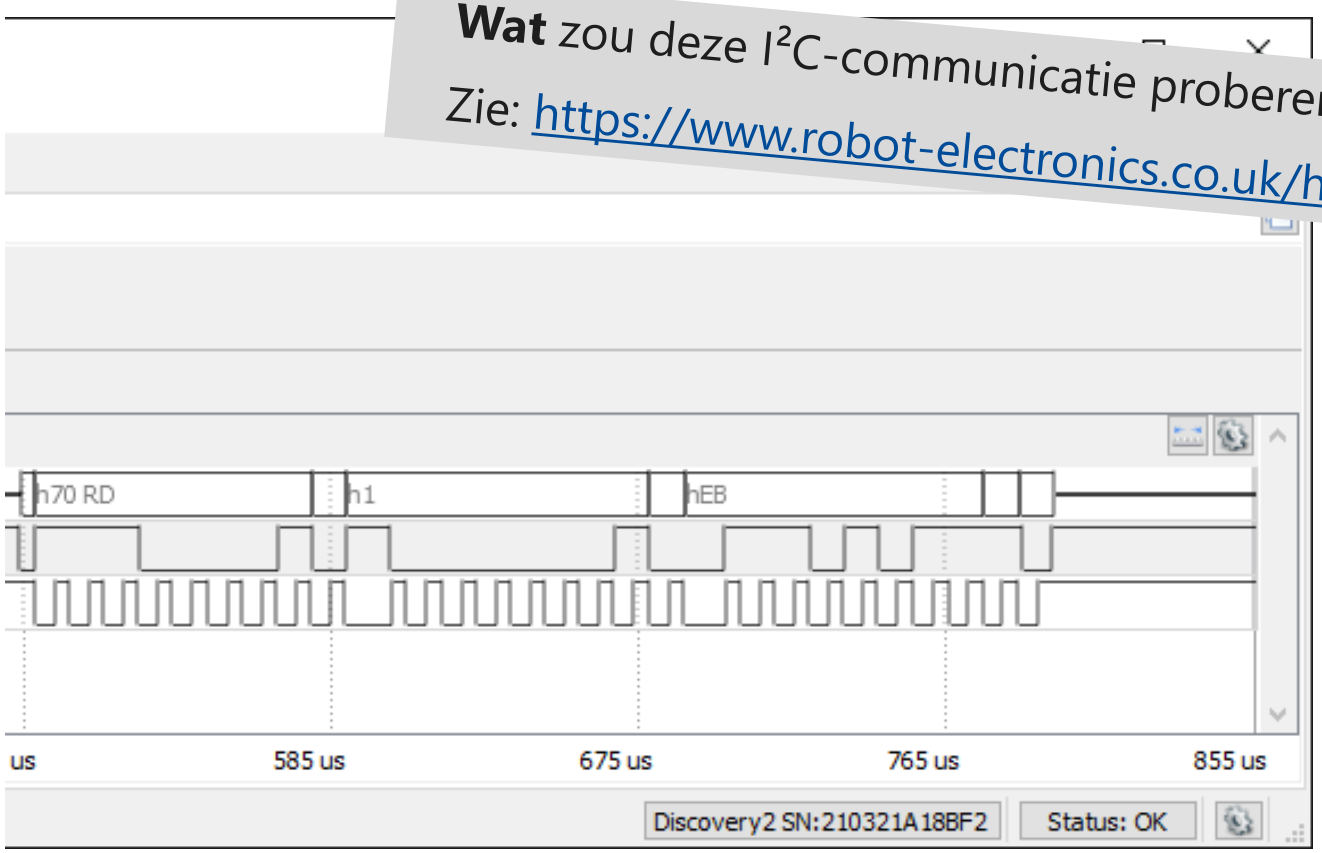
Zie: https://www.robot-electronics.co.uk/htm/srf02techI2C.htm



I²C

- Toepassingsvoorbeeld: ultrasonic **range finder** (SRF-02)

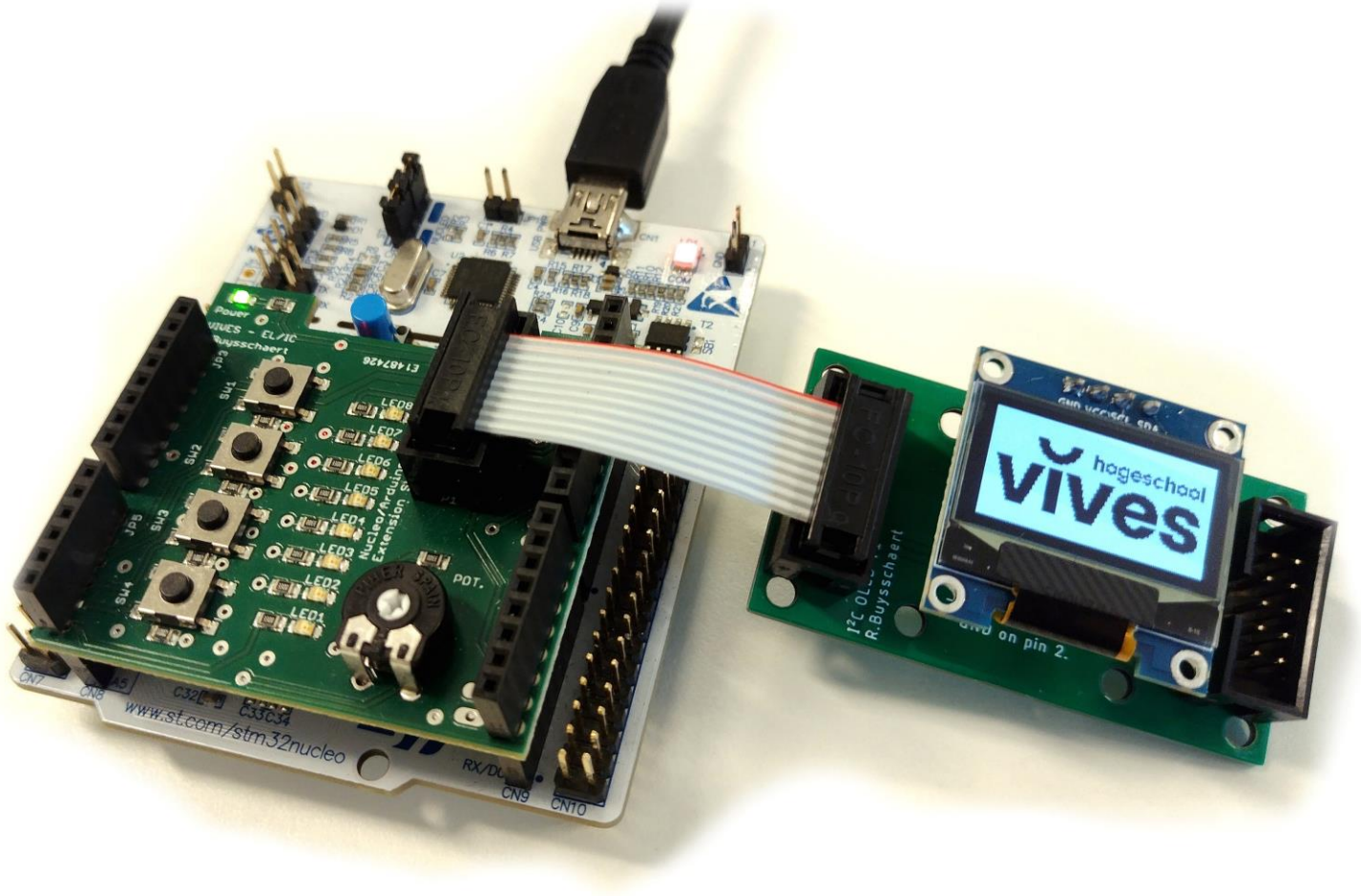
Wat zou deze I²C-communicatie proberen te **verwezenlijken**?
Zie: https://www.robot-electronics.co.uk/htm/srf02techI2C.htm



0x01EB = 491 cm

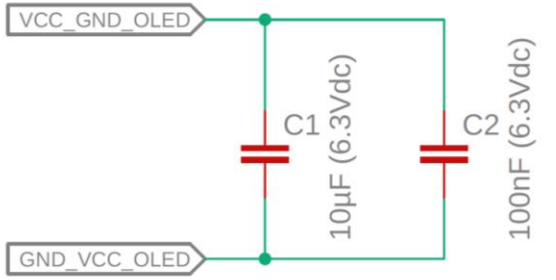
I²C

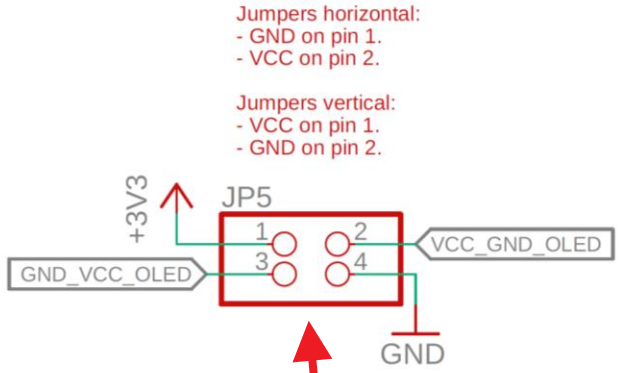
- Toepassingsvoorbeeld: **OLED-display** op I²C

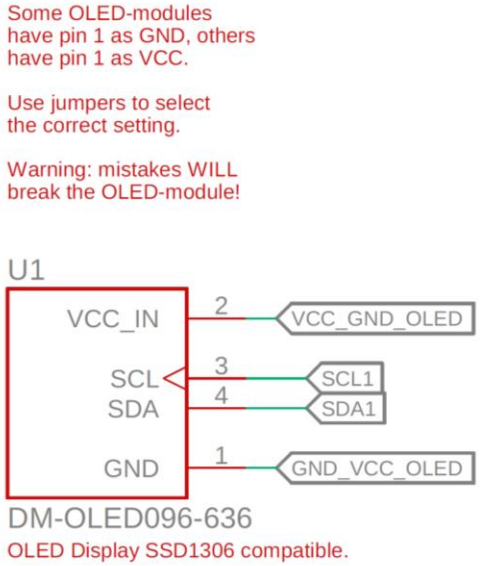


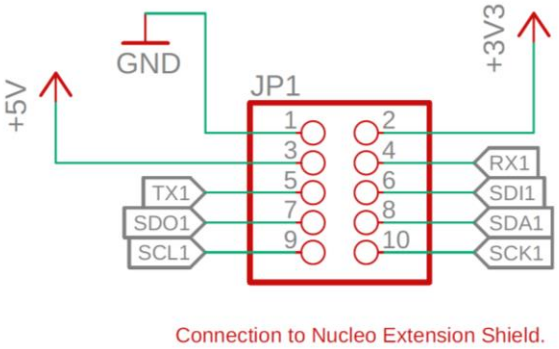
I²C

- Toepassingsvoorbeeld: **OLED-display** op I²C









OPM: **jumpers** bepalen waar/hoe de voedingsspanning aangeleverd wordt.

I²C

- Toepassingsvoorbeeld: **OLED-display** op I²C

```
// OLED-scherm wissen.
OLED_FillScreen(0);

// Welkom...
OLED_StringToPage("VIVES - Kortrijk", 1, false);
OLED_StringToPage("Elektronica - ICT", 3, false);
OLED_StringToPage("R.Buysschaert", 5, false);
//TODO: versie aanpassen indien nodig.
OLED_StringToPage("OLED USB-display V1.0", 7, false);
WaitForMs(2000);

// OLED-scherm wissen.
OLED_FillScreen(0);
```

I²C

- Toepassingsvoorbeeld: **OLED-display** op I²C

```
204    // Verwijzen naar de juiste page.
205    OLED_SetPageStartAddress(pageNumber);
206
207    // Naar het begin van de page gaan.
208    OLED_SetColumnStartAddress(0);
209
210    // Data verzenden (met extra control byte).
211    I2C1WriteArray(OLED_SSD1306_ADDRESS, i2cData, (textIndexer * CHARACTER_WIDTH + 1));
212  }
```

Zie: “i2c.c” of volgende slide ;-).

hogeschool vives

I²C

- Toepassingsvoorbeeld: **OLED-display** op I²C

```
// Schrijf array naar een bepaald I²C-apparaat.
void I2C1WriteArray(uint8_t deviceAddress, uint8_t* data, uint8_t length)
{
  uint8_t index = 0;

  // Schrijfactie.
  I2C1->CR2 &= ~I2C_CR2_RD_WRN;

  // Slave address waarmee gecommuniceerd moet worden invullen.
  I2C1->CR2 &= 0xFFFFFF00;                          // Slave address resetten.
  I2C1->CR2 |= (deviceAddress << 1);                // Adres klaarzetten.

  // Aantal te versturen bytes, waarna een stop gegenereerd moet worden (TC vlag wordt gezet).
  I2C1->CR2 &= ~I2C_CR2_NBYTES;                     // NBYTES wissen.

  // Aantal te verzenden bytes.
  I2C1->CR2 |= (length << 16);                      // x bytes te versturen, dus x in NBYTES

  // Wachten tot vorige communicatie is afgelopen.
  while((I2C1->ISR & I2C_ISR_BUSY) == I2C_ISR_BUSY);
```

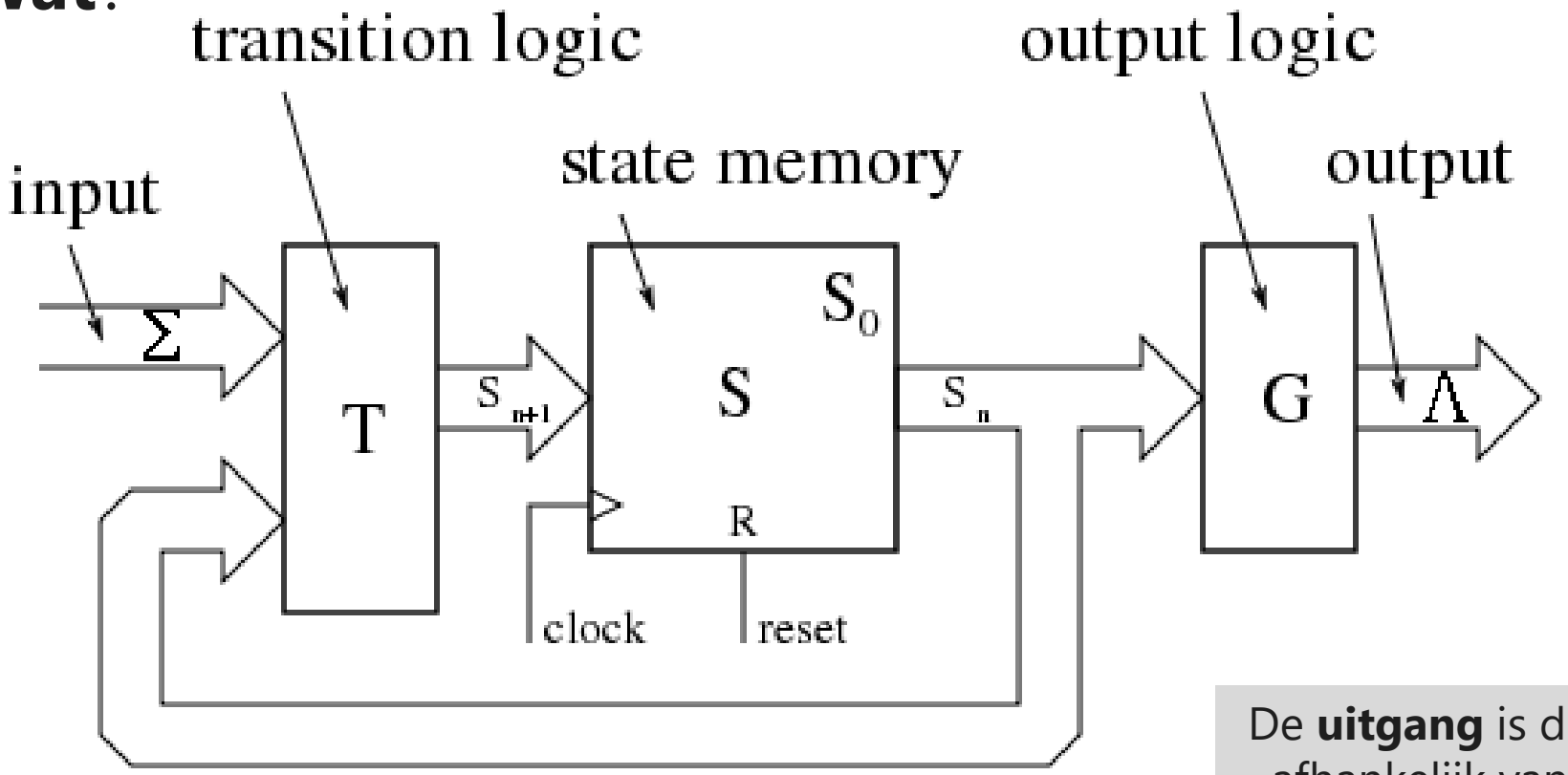
State machines

State machines

- **Wat**?
  - Heel veel toestellen hebben een **wel bepaald aantal toestanden** waarin ze zich kunnen bevinden.
  - Bijvoorbeeld: **verkeerslichten**, snoepverkoopautomaat, lift, doorgeefsluis, …
  - Als we een **grafisch** overzicht maken van die toestanden, kunnen we gemakkelijk **overgaan tot programmeren** van een '**state machine**'.
  - **Voordeel**: sterke koppeling van grafische voorstelling naar je programma. **Tekening ok = programma ok**!

State machines

- **Wat**?



De **uitgang** is dus niet enkel afhankelijk van de huidige ingangen, maar **ook van de vorige toestand**…

Bron: https://upload.wikimedia.org/wikipedia/commons/1/1f/Moore-Automat-en.svg

State machines

- Voorbeeld: **spoorwegovergang**

We **vereenvoudigen** wel wat en gaan ervan uit dat er maar **één sensor vóór en één sensor ná** de overweg ligt…

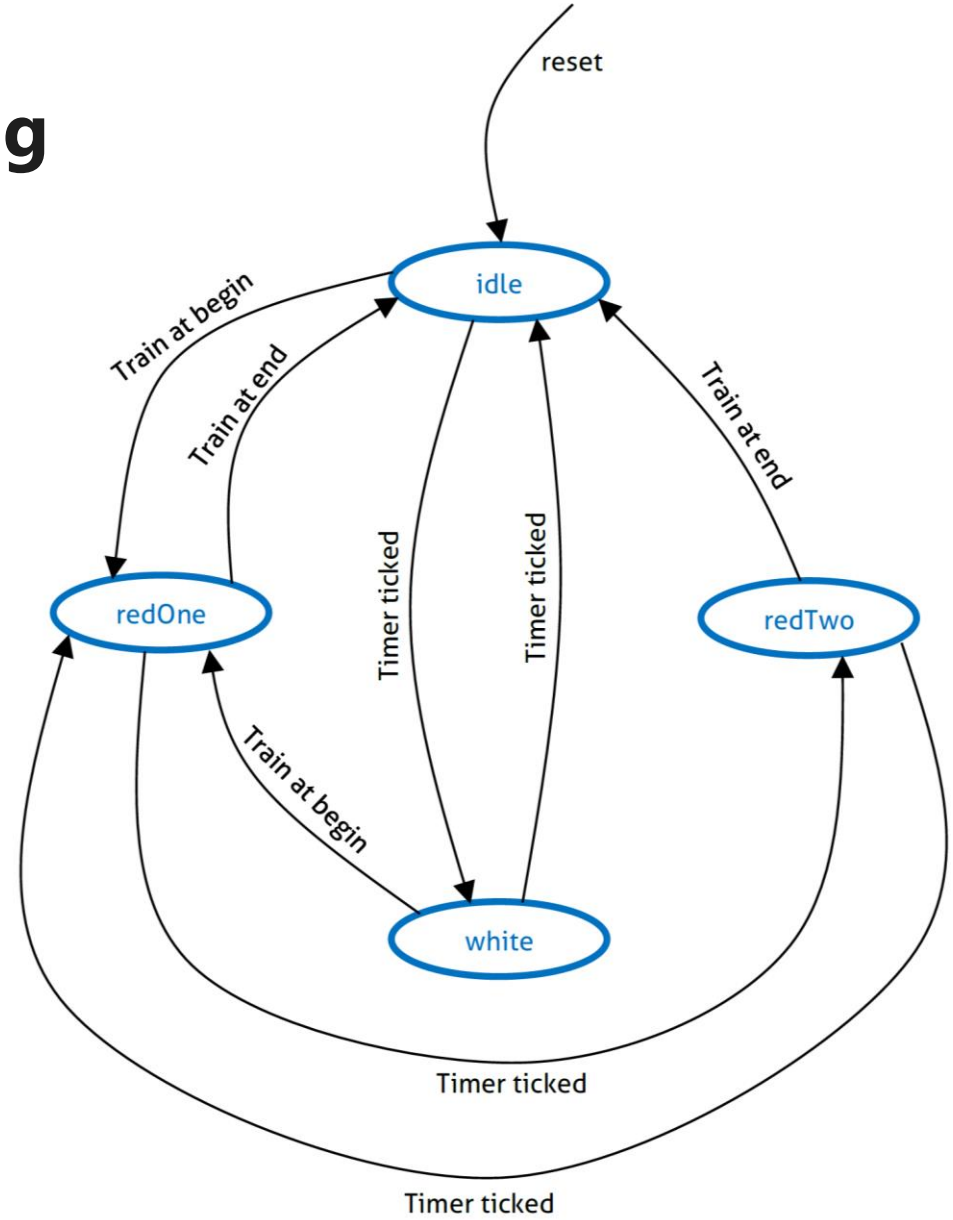
Verder zijn er wel twee rode lichten en één wit licht.

Bron: https://images.vrt.be/width960/2019/04/02/23897aed-5514-11e9-abcc-02b7b76bf47f.jpg

State machines

- Voorbeeld: **spoorwegovergang**

Zie ook: “State machine railway.pdf”.



State machines

- Voorbeeld: **spoorwegovergang**

```
24 // Bedenk een nieuwe enumeratie (= zelf bedachte opsomming).
25 enum States {idle = 0, white, redOne, redTwo};
26 // Maak gebruik van de enumeratie.
27 enum States currentState, nextState;
```

Gebruik **enumeraties** om zelf bedachte variabelen aan te maken, met **daarin de opsomming** van de gewenste mogelijkheden/states.

State machines

- Voorbeeld: **spoorwegovergang**

```
54      // Overloop de state machine.
55      switch(currentState)
56      {
57        // Alle lampen uit. Toestand na reset.
58        case idle:
59          ByteToLeds(0);
60          if(SW1Active())
61            nextState = redOne;
62          else
63            if(timerTicked)
64              nextState = white;
65          break;
66
67        // Witte lamp aan.
68        case white:
69          ByteToLeds(0b00011000);
70          if(SW1Active())
71            nextState = redOne;
72          else
73            if(timerTicked)
74              nextState = idle;
75          break;
```

Gebruik een **switch** statement om alle mogelijke toestanden te realiseren.

```
102     // Is er een nieuwe state, doe een update.
103     if(currentState != nextState)
104     {
105       currentState = nextState;
106
107       sprintf(text, "Next state is: %d.\r\n", nextState);
108       StringToUsart2(text);
109     }
```

Indien nodig, **update** de huidige **toestand**...

State machines

- **En nu**?
  - Probeer **zelf** enkele state machines uit te werken.
  - Dat kan **bijvoorbeeld** met:
    - een liftsturing.
    - een verkeerslichtsturing.
    - een beveiliging voor een papiersnijmachine.
    - een fancy richtingaanwijzer voor een auto.
    - een sturing voor een tweetrapsventilator.
    - …

Tip: de kans is super groot dat er een state machine gevraagd wordt op het **oefeningen-examen**…

Your turn now!

- Is er nog **wat resterende** tijd?

  - Demo ‘**System Viewer**’ in debug mode.
  - Vragen?
  - Feedback voorbije theorieopdrachten?
  - Maak **Theorieopdracht 8 – structs en pointers.**

  - Ga **zelf aan de slag** om …
  - Samen opdrachten/oefeningen maken?